<Learning Management system>

Software Requirements Specification

<1.0>

<25/05/2025>

Umar Farooq 09-131242-088
Ahtesham Khan 09-131242-014

Prepared for
Introduction to software engineering course
Instructor: Ma’am Rafia, Ph.D. Spring 2025

Revision History

Date	Description	Author	Comments
<date>	<Version 1>	<Your Name>	<First Revision>

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Lead Software Eng.	
	A. David McKinnon	Instructor, CptS 322	

Table of Contents

1. Introduction

This Software Requirements Specification (SRS) document outlines the requirements for the Learning Management System (LMS), a web and mobile-based platform designed to facilitate online learning and course management for students, instructors, and administrators. The purpose of this document is to provide a comprehensive and clear description of the system's functional and non-functional requirements, ensuring that software engineers have all necessary information to design, implement, and test the LMS effectively. The document is intended for stakeholders including developers, testers, project managers, instructors, and administrative staff involved in the development and use of the LMS.

1.1 Purpose

The purpose of this SRS is to:

- Define the scope, features, and constraints of the LMS.
- Provide a detailed description of system requirements for developers and testers.
- Serve as a reference for stakeholders to ensure the final product meets their expectations.
- Reduce the risk of project failure by identifying and resolving potential issues early in the development process.
- Guide the design, development, testing, and deployment phases with clearly defined requirements.
- Support project management activities such as timeline estimation, resource planning, and task allocation.

1.2 Scope

This subsection defines the boundaries, functionalities, and objectives of the Learning Management System (LMS) software product.

i. **Software Product Identification**

The software to be developed is a web and mobile-based Learning Management System (LMS) designed for educational institutions, instructors, and students**.**

ii. **System Capabilities and Limitations**

What the LMS will do:

- **Instructors** to create, upload, and manage course materials (e.g., lecture notes, quizzes, assignments), mark attendance, grade assignments, and provide feedback.
- **Students** to enroll in and drop courses, access course materials, submit assignments, view grades and attendance records, and receive personalized course recommendations based on past enrollments and interests.
- **Administrators** to manage user accounts, system settings, course configurations, and oversee processes like attendance and result management.
- **Security and Compliance** features, including **AES-256** encryption for data security and compliance with **GDPR** and **FERPA** regulations.
- **User Interface** features, such as a dashboard with relevant statistics, dark/light mode support, a structured navigation bar, and a search bar for quick access to courses and materials.
- **Payment Processing** for premium courses via secure methods like PayPal and credit cards.

What LMS will not do:

- **Live video conferencing** (e.g., Zoom integration is out of scope).
- **Third-party integrations** beyond basic payment gateways (e.g., PayPal for premium courses).
- **Offline functionality** (users require internet access for all features).

iii. Application and objectives:

The LMS is designed for educational institutions to streamline online learning and course administration. It will be accessible on both mobile and desktop devices, supporting a scalable cloud-based architecture to accommodate future growth.

1. Benefits, Objectives, and Goals

Benefits: The LMS will enhance the educational experience by providing a centralized platform for course management, real-time progress tracking, and secure data handling. It will improve accessibility and user engagement through customizable interfaces and personalized course recommendations.

Objectives:

- Enable instructors to upload and manage course materials (e.g., videos, PDFs, quizzes) with a maximum upload time of 10 seconds for files up to 100 MB.
- Allow students to enroll in courses and access materials within 2 seconds of page load under normal conditions.
- Provide real-time grade updates and attendance records with 99.9% system uptime.
- Ensure data security through **AES-256** encryption and compliance with **GDPR** and **FERPA** regulations.
- Support at least 500 concurrent users without performance degradation.
- Deliver parameter-driven, user-definable performance reports exportable in **CSV**/**PDF** formats within 5 seconds.

Goals:

- Achieve a user-friendly interface with customizable profiles (e.g., profile pictures, bios) and notification settings (email, SMS, in-app alerts).
- Implement course recommendations based on user enrollment history and interests, updated within 1 second of profile access.
- Ensure system scalability to support future expansion, targeting a 20% increase in user capacity within the first-year post-deployment.

2. **Consistency with Higher-Level Specifications**

   This SRS aligns with the high-level requirements outlined in the project assignments (ISE 1, ISE 2, ISE 3), which emphasize a scalable, secure, and user-friendly LMS. The system requirements focus on functional capabilities (e.g., course management, progress tracking) and non-functional attributes (e.g., 99.9% uptime, 2-second page load time), consistent with the course's objectives for a robust educational platform.

1.3 Definitions, Acronyms, and Abbreviations

This subsection provides definitions for key terms, acronyms, and abbreviations used throughout the Software Requirements Specification (SRS) for the Learning Management System (LMS). These definitions ensure clarity and consistent interpretation for all stakeholders, including developers, testers, instructors, administrators, and other users. Additional details, if required, are provided in Appendix A.1 of this SRS.

- **LMS**: Learning Management System, the web and mobile-based software platform designed to facilitate online learning and course management for educational institutions.
- **AES-256**: Advanced Encryption Standard with a 256-bit key, a cryptographic algorithm used to secure user data within the LMS.
- **GDPR**: General Data Protection Regulation, a European Union regulation governing data protection and privacy.
- **FERPA**: Family Educational Rights and Privacy Act, a U.S. federal law protecting the privacy of student education records.
- **Concurrent Users**: The maximum number of users actively interacting with the system at the same time.
- **Uptime**: The percentage of time the system is operational and accessible (target: 99.9%).
- **Two-Factor Authentication (2FA):** An extra security layer requiring users to verify identity via two methods (e.g., password + SMS code).
- **FR**: Functional Requirement, a specific feature or capability that the LMS must provide, such as course enrollment or grade updates.
- **NFR**: Non-Functional Requirement, a quality attribute or constraint of the LMS, such as performance, security, or scalability.
- **DFD**: Data Flow Diagram, a visual representation of how data moves between external entities, processes, and data stores within the LMS.
- **Use Case**: A description of how a user (e.g., Student, Instructor, Admin) interacts with the LMS to achieve a specific goal, such as uploading an assignment or generating a report.
- **CSV**: Comma-Separated Values, a file format for exporting reports (e.g., student performance reports).
- **PDF**: Portable Document Format, a file format for exporting and viewing course materials and reports.
- **UI**: User Interface, the visual and interactive components of the LMS (e.g., dashboard, navigation bar).
- **API**: Application Programming Interface, used for integrating payment gateways or authentication services with the LMS.

1.4 References

1.4.1: ISE 1: LMS Project Overview and Initial Requirements

- Report Number: ISE-001
- Date: March 15, 2025
- Publishing Organization: University Software Engineering Department
- Description: Outlines scope, objectives, requirements, constraints. Referenced in SRS 1, 3.2, 3.5, 3.6, 3.7, A.1.
- Source: GitHub /docs/ISE-001_LMS_Overview.pdf

1.4.2: ISE 2: LMS Stakeholder Requirements and Use Cases

- Report Number: ISE-002
- Date: April 10, 2025
- Publishing Organization: University Software Engineering Department
- Description: Details stakeholder needs, use cases. Referenced in SRS 3.3, 3.4, 3.6, 3.9, A.2.
- Source: GitHub /docs/ISE-002_Stakeholder_Reqs.pdf

1.4.3: ISE 3: LMS Data and Architecture Specifications

- Report Number: ISE-003
- Date: April 25, 2025
- Publishing Organization: University Software Engineering Department
- Description: Specifies data stores, architecture. Referenced in SRS 3.4, 3.8, 4.
- Source: GitHub /docs/ISE-003_Data_Arch.pdf

1.4.4: SRS Template for Software Engineering Projects

- Report Number: SE-SRS-Template-2025
- Date: January 10, 2025
- Publishing Organization: University Software Engineering Department
- Description: Guides SRS structure. Referenced throughout SRS.
- Source: https://se.university.edu/resources/SRS_Template_2025.pdf, GitHub /docs/SE-SRS-Template-2025.pdf.

1.4.5: GDPR: General Data Protection Regulation

- Report Number: Regulation (EU) 2016/679
- Date: April 27, 2016 (effective May 25, 2018)
- Publishing Organization: European Parliament and Council
- Description: Mandates data protection. Referenced in SRS 3.7 (DC-1), 3.8 (DBR-4, DBR-6), 3.9 (OR-4).
- Source: https://eur-lex.europa.eu/eli/reg/2016/679/oj, GitHub /docs/references/GDPR.pdf.

1.4.6: FERPA: Family Educational Rights and Privacy Act

- Report Number: 20 U.S.C. § 1232g
- Date: August 21, 1974 (amended January 7, 2022)
- Publishing Organization: U.S. Department of Education
- Description: Protects student records. Referenced in SRS 3.7 (DC-1), 3.8 (DBR-4, DBR-6), 3.9 (OR-4).
- Source: https://www2.ed.gov/policy/gen/guid/fpco/ferpa/index.html, GitHub /docs/references/FERPA.pdf.

1.4.7: WCAG 2.1: Web Content Accessibility Guidelines

- Report Number: W3C Recommendation
- Date: June 5, 2018
- Publishing Organization: World Wide Web Consortium (W3C)
- Description: Ensures UI accessibility. Referenced in SRS 3.9 (OR-5).
- Source: https://www.w3.org/TR/WCAG21/, GitHub /docs/references/WCAG21.pdf.

1.5 Overview

This subsection provides a high-level summary of the Software Requirements Specification (SRS) document's structure and content organization.

i. Content of SRS

- **General Description**: An overview of the LMS, including its context, major functions, user characteristics, constraints, and assumptions that influence the requirements.
- **Specific Requirements**: Detailed functional requirements (e.g., course enrollment, assignment submission, grade updates), non-functional requirements (e.g., performance, security, scalability), external interface

requirements (e.g., user, hardware, software, and communication interfaces), use cases, class/object models, and logical database requirements.

- **Analysis Models**: Visual representations such as sequence diagrams, data flow diagrams (DFDs), and state-transition diagrams (STDs) to illustrate system processes and support requirement traceability.
- **Change Management Process**: Procedures for updating the SRS in response to changes in project scope or requirements.
- **Appendices**: Supplementary information, including additional definitions, conceptual documents, and supporting materials to enhance understanding of the LMS requirements.

ii. Document Organization

- **Introduction** Purpose, scope, definitions, and references.
- **General Description:** High-level system perspective, user characteristics, and operational constraints.
- **Specific Requirements:** Functional Requirements: Features grouped by user roles (student, instructor, admin).
- **Non-Functional Requirements:** Security, performance, and scalability criteria.
- **Use Cases / Diagrams:** Interaction flows (e.g., "Submit Assignment").
- **Analysis Models:** Visual aids (e.g., UML diagrams, data flow) to clarify complex requirements.
- **Appendices:** Supporting documents (e.g., full glossary, compliance certificates).

2. General Description

This section describes the general factors that influence the Learning Management System (LMS) and its requirements, providing context to make the specific requirements in Section 3 easier to understand without stating detailed requirements.

2.1 Product Perspective

The Learning Management System (LMS) is a standalone, cloud-based platform designed to facilitate online learning and course management within educational institutions. It operates as an integrated system that connects students, instructors, and administrators through a web and mobile interface. The LMS interfaces with external systems, such as payment gateways (e.g., PayPal, credit cards) for premium course transactions and an authentication service for secure user access. Unlike subsystems embedded within larger platforms, the LMS is a self-contained product that can integrate with institutional databases for user and course data.

Relationship with other systems:

External system/component	Interaction with LMS	Dependency criticality
Institutional user database	Syncs students/instructor profiles	High (Required for login)
Payment gateway	Processes course enrolment fees	High (for enrolling in course)
Email service	Sends notification (e.g. deadline)	Low
Cloud storage	Hosts course materials (e.g. Assignments)	High (Scalability requirement)

2.2 Product Functions

The Learning Management System (LMS) will provide the following core functionalities, organized by key user roles and system capabilities:

- **Course Management**: Enables instructors to create, upload, and manage course materials (e.g., videos, PDFs, quizzes, lecture notes) and allows students to enroll in or drop courses dynamically. The system supports course recommendations based on students' past enrollments and interests.
- **Progress Tracking and Evaluation**: Facilitates automated grading of quizzes and assignments, provides real-time updates to student grades, and generates detailed performance reports in CSV or PDF formats for students and instructors.
- **Attendance Management**: Allows instructors to manually mark attendance for class sessions and enables students to view their attendance records in real time.

- **Feedback and Reviews**: Permits students to rate courses and submit reviews upon course completion to improve course quality and inform future students.
- **Personalization**: Supports customizable user profiles (e.g., profile pictures, bios), user-defined notification settings (e.g., email, SMS, in-app alerts), and learning goal tracking with progress reminders for students.
- **Administrative Oversight**: Provides administrators with tools to manage user accounts, configure system settings, oversee attendance and result management, and generate system-wide reports on course and user activity.
- **Security and Payment Processing**: Ensures data security through AES-256 encryption and compliance with GDPR and FERPA regulations, while supporting secure payment processing for premium courses via PayPal or credit cards.

2.3 User Characteristics

This subsection outlines the key attributes of the LMS's target user groups, focusing on characteristics that directly influence system design and requirements. Understanding these user profiles ensures the LMS meets actual needs while maintaining usability.

Students

Students are the main users, usually from high school to university level. Most are comfortable with technology but have different skill levels. They want an easy-to-use LMS that works well on both phones and computers. They use it to enroll in courses, watch videos, read PDFs, submit assignments, check grades, and track attendance. Students like features that give quick access (like fast-loading pages), personalized suggestions (such as recommended courses), and clear messages (like “Upload successful”).

Instructors

Instructors are teachers or professors. Most are good with technology, but some may need a bit of help. They want simple tools to upload lessons (videos, notes, quizzes), grade work, take attendance, and see how students are doing. They need organized ways to manage course content and export grades or reports in PDF or Excel. Instructors prefer real-time updates and easy dashboards to save time and reduce extra work.

Administrators

Admins are school or university staff who manage the whole system. Some are tech-savvy, while others may rely on IT support. They need strong tools to manage

users, set up the system, follow privacy laws (like GDPR), and create reports. Admins also handle system security, so they need features like two-factor login and cloud storage. They need detailed logs and system monitoring tools to keep everything running smoothly.

2.4 General Constraints

This subsection outlines the general constraints that limit the developer's options for designing the Learning Management System (LMS). The following constraints are tailored to the LMS project, focusing on practical limitations that shape the system's design.

- Time Limit: The LMS must be developed and ready for use within 6 months. This short timeline means developers must focus on essential features, like course management and user logins, and avoid adding complex extras that take too long to build.
- Data Privacy Rules: The system must follow strict privacy laws to protect user information, requiring strong data encryption (like AES-256) and secure storage. This limits the choice of tools to those that support high-security standards and prevents using simpler, less secure options.
- Device Compatibility: The LMS must work smoothly on both phones and computers, supporting popular browsers (e.g., Chrome, Firefox) and mobile systems (iOS, Android). This requires a flexible, responsive design, ruling out solutions that only work on specific devices.
- Speed and Reliability: The system must load pages in 2 seconds or less and stay online 99.9% of the time. These performance goals limit developers to fast, efficient coding methods and reliable hosting options, avoiding slower or less stable technologies.
- User Capacity: The LMS must handle at least 500 users at once without slowing down and use a cloud-based setup for future growth. This restricts developers to cloud-friendly tools and prevents using basic, non-scalable systems.
- Security Needs: The system must use strong encryption for user data and secure login methods, like two-factor authentication. This limits developers to secure, industry-standard technologies and excludes less protected alternatives.

- External Services: The LMS depends on outside services, like payment systems (e.g., PayPal) for course fees and an authentication service for logins. This forces developers to design the system to work with these services' specific rules and setups, reducing flexibility.

2.5 Assumptions and Dependencies

This subsection lists the assumptions and dependencies that affect the requirements of the Learning Management System (LMS). These factors are not design constraints but external conditions or resources that, if changed, would necessitate updates to the SRS requirements.

Assumptions

- **User Device Access**: Users (students, instructors, administrators) have access to modern devices, such as smartphones or computers, running recent versions of web browsers (e.g., Chrome, Firefox) or mobile operating systems (iOS, Android), to interact with the LMS. If users lack compatible devices, requirements for accessibility and interface design may need adjustment.
- **Internet Connectivity**: Educational institutions and users have reliable, high-speed internet connections to support real-time interactions, such as accessing course materials or submitting assignments. Poor connectivity could necessitate changes to performance or offline functionality requirements.
- **Institutional Data Availability**: User data (e.g., student IDs, course records) and course information are readily available from institutional databases for integration into the LMS. If this data is incomplete or inaccessible, requirements for data import and user management may require revision.
- **User Feedback Availability**: Stakeholders (students, instructors, administrators) will provide timely feedback during the Agile development process to refine features like course recommendations or dashboard usability. Lack of feedback could impact requirements for user interface and personalization features.

Dependencies

- **Payment Gateway Services**: The LMS relies on third-party payment gateways (e.g., PayPal, credit cards) to process transactions for premium courses. If these services change their APIs, become unavailable, or impose new restrictions, requirements for payment processing and integration may need updates.
- **Authentication Service**: An external authentication service is used for secure user login and registration, including support for two-factor authentication

(2FA). Changes to this service's functionality or availability could affect requirements for user authentication and security.

- **Cloud Infrastructure**: The LMS depends on a cloud-based infrastructure (e.g., AWS, Azure) to ensure scalability and 99.9% uptime. If the chosen cloud provider experiences disruptions or imposes new limitations, requirements for hosting, performance, or scalability may require revision.
- **Regulatory Compliance Support**: Compliance with GDPR and FERPA regulations depends on the institution's existing policies and data handling procedures. If these policies change or new regulations are introduced, requirements for data privacy and security may need adjustment.
- **Development Tools**: The project assumes the availability of open-source or low-cost development tools and frameworks suitable for a student team. If these tools become unavailable or insufficient, requirements for system implementation or testing may need to be modified.

3. Specific Requirements

This section provides the detailed requirements for the Learning Management System (LMS), guiding the software's design, implementation, and testing. These requirements build on the customer requirements outlined in Section 2. The requirements are organized for clarity and accessibility, avoiding over-constraining the design to maintain flexibility for developers.

3.1 External Interface Requirements

This subsection details the external interface requirements for the LMS, covering user, hardware, software, and communications interfaces. These requirements ensure the system interacts effectively with users and external systems, supporting the functionality.

3.1.1 User Interfaces

The LMS user interfaces must provide intuitive and responsive access for students, instructors, and administrators.

ID	DESCRIPTION	PRIORITY	VERIFICATION METHOD
3.1.1.1	Toggleable dark/light modes for user comfort.	Medium	Mode switch updates in ≤1s.
3.1.1.2	Search bar for courses/materials, results in ≤3s.	Medium	Keyword search returns result in ≤3s.
3.1.1.3	The LMS shall provide a **responsive web interface** compatible with Chrome, Firefox, and Edge (latest versions).	High	Cross-browser testing

3.1.1.4	Role-specific dashboard showing course progress (students), performance (instructors), or system metrics (admins).	High	Dashboard displays correct data in ≤2s.

3.1.2 Hardware Interfaces

The LMS is a cloud-based system with minimal hardware interface requirements, but it must support specific hardware to ensure accessibility and performance.

ID	DESCRIPTION	PRIORITY	VERIFICATION METHOD
3.1.2.1	Operates on devices with ≥2GB RAM, iOS 15+/Android 10+.	High	Functionality on minimum-spec devices, loads in ≤2s.
3.1.2.2	No specialized hardware (e.g., scanners) shall be required for core functions.	Low	Requirement review

3.1.3 Software Interfaces

The LMS interacts with external software systems to support its functionality, particularly for authentication and payment processing.

ID	DESCRIPTION	PRIORITY	VERIFICATION METHOD
3.1.3.1	Integrates with authentication service for login/2FA via OAuth 2.0.	High	Integrates with authentication service for login/2FA via OAuth 2.0.
3.1.3.2	Integrates with PayPal/Stripe for secure payments via HTTPS.	High	Transaction completes in ≤10s.
3.1.3.3	Uses MySQL/PostgreSQL for data storage, SQL queries.	High	Data retrieval in ≤1s.
3.1.3.4	Email notifications shall be sent via Gmail or Outlook	Low	Email delivery logs

3.1.4 Communications Interfaces

The LMS relies on network-based communication protocols to ensure secure and reliable interactions.

ID	DESCRIPTION	PRIORITY	VERIFICATION METHOD
3.1.4.1	Uses HTTPS (TLS 1.2+) for secure web communication.	High	All requests use HTTPS, data encrypted.
3.1.4.2	Sends notifications via SMTP for email/SMS.	Medium	Notifications sent/received in ≤10s.

3.2 Functional Requirements

This section describes the specific features of the Learning Management System (LMS) software project. The functional requirements detail the core capabilities the system must provide to meet user needs.

3.2.1 <Course Enrollemnt>

3.2.1.1 Introduction

The LMS shall allow students to enroll in available courses, enabling access to materials, assignments, and progress tracking.

3.2.1.2 Inputs

- **Student Credentials**: Student must use Valid username/password, which will be verified with 2FA if enabled.
- **Course Selection**: Student will select course from catalogue/search.
- **Enrollment Confirmation**: Student clicks “Enroll” button.
- **Payment Details**: Credit card/PayPal for courses.

3.2.1.3 Processing

- Verify credentials via authentication service.
- Retrieve available courses, check eligibility/limits.
- For courses, process payment via gateway.
- Update database with student enrollment.
- Send confirmation notification (email/in-app).

3.2.1.4 Outputs

- **Updated Course Roster**: Student added to course in database.
- **Access Grant**: Student accesses course materials/assignments.
- **Confirmation Notification**: Email/in-app message with course details.
- **Success Message**: Interface shows “Successfully enrolled” in ≤2s.
- **Payment Receipt** (if applicable): Email/interface receipt.

3.2.1.5 Error Handling

- **Invalid Credentials**: Show “Invalid username/password,” prompt retry.
- **Course Unavailable**: Show “Course not available,” suggest alternatives.
- **Payment Failure**: Show “Payment failed,” allow retry/cancel.
- **System Timeout**: Show “Request timed out,” revert changes.
- **Duplicate Enrollment**: Show “Already enrolled,” prevent duplication.

3.2.2 <Assignment submission>

3.3 Use Cases

This section continues the use case descriptions for the Learning Management System (LMS), detailing how users interact with the system to achieve specific goals

3.3.1 Enroll in a Course

Description: A student enrolls in an available course, gaining access to materials, assignments, and tracking.

Actors:

- **Primary**: Student.
- **Secondary**: Authentication Service, Payment Gateway, LMS Database.

Pre-conditions:

- Student is registered with valid credentials.
- Student is logged in.
- At least one course is available.

Post-Conditions

Basic Flow

- Student navigates to course catalog or searches.
- System displays courses with details.
- Student selects course, clicks “Enroll.”
- System verifies credentials.
- System checks course availability.
- System updates database with enrollment.
- System grants access to course features.
- System sends confirmation notification.
- System displays “Successfully enrolled” message.

Assumptions

- Authentication/payment services respond in ≤5s.
- Database updated with course availability.

Verification

- Test enrollment completes in ≤5s, materials accessible, notification sent.
- Verify database updates.

3.3.2 Submit Assignment

Description: A student submits an assignment for an enrolled course, allowing instructor grading**.**

Priority: High

Actors:

- **Primary:** Student.
- **Secondary:** Authentication Service, LMS Database, Notification Service.

Pre-conditions:

- Student is registered with valid credentials.
- Student is logged in and enrolled in the course.
- Course has an open assignment.
- Student has a file (≤100 MB) ready.

Post-Conditions:

- Assignment file/comments stored in database.
- Assignment status updated to "Submitted."
- Student receives confirmation notification.
- Instructor notified of new submission.

Basic Flow:

- Student navigates to course's "Assignments" section.
- System displays assignments with details.
- Student selects open assignment, clicks "Submit."
- System verifies credentials and enrollment.
- System prompts for file upload and optional comment.
- Student uploads file (e.g., PDF) and comment, confirms.
- System validates file format/size/security.
- System stores file/comment in database.
- System updates status to "Submitted" with timestamp.
- System sends confirmation notification to student.
- System notifies instructor of submission.
- System displays "Assignment submitted" message.

Assumptions:

- Authentication/notification services respond in ≤5s.
- Database updated with assignment statuses.

Verification:

- Test submission of 5 MB PDF completes in ≤5s, file stored, notifications sent.
- Verify database updates.

3.4 Classes / Objects

This section describes the key classes and objects in the Learning Management System (LMS), providing a high-level view of their structure to support the functional requirements and use cases.

3.4.1 <Course>

3.4.1.1 Attributes:

- **courseID**: String
- **title**: String
- **description**: String
- **instructorID**: String
- **enrollmentLimit**: Integer
- **startDate**: Date
- **endDate**: Date
- **materials**: List

3.4.1.2 Functions:

- **createCourse(title, description, instructorID, enrollmentLimit, startDate, endDate)**: Void
  - Initialize new course.
  - Input: title, description, instructorID, enrollmentLimit, startDate, endDate.
  - Output: Creates Course in database.
- **updateCourseDetails(title, description, enrollmentLimit, startDate, endDate)**: Void
  - Update attributes (except courseID, instructorID).
  - Input: title, description, enrollmentLimit, startDate, endDate.
  - Output: Updates Course in database.
- **addMaterial(filePath)**: Boolean
  - Add material if uploaded by instructor.
  - Input: filePath (String).
  - Output: True if added, false otherwise.
- **removeMaterial(filePath)**: Boolean
  - Remove material if requested by instructor.
  - Input: filePath (String).

    - Output: True if removed, false otherwise.
- **enrollStudent(studentID)**: Boolean
    - Add student if course is “Open” and under limit.
    - Input: studentID (String).
    - Output: True if enrolled, false if full/payment fails.
- **getCourseDetails()**: Course
    - Retrieve all course attributes.
    - Input: None.
    - Output: Course object.

Reference to functional requirements and/or use cases

- **FR-1**: Course Enrollment (3.2.1). Supports enrollment via enrollStudent.
- **UC-1**: Enroll in a Course (3.3.1). Supports availability check, roster update, course selection.
- **UC-2**: Submit Assignment (3.3.2). Provides course context (materials, status).

3.4.2 <User>

3.4.2.1 Attributes:

- **userID: String**
- **username: String**
- **password: String**
- **role: String**
- **email: String**
- **fullName: String**
- **Description: String**
- **enrolledCourses: List**

3.4.2.2 Function:

- **createUser(username, password, role, email, fullName)**: Void
    - Initialize new user, assign userID, password.
    - Input: username, password, role, email, fullName.
    - Output: Creates User in database.
- **updateUserProfile(fullName, profilePicture, bio, notificationSettings)**: Void
    - Update profile attributes (except userID, username, role, passwordHash).
    - Input: fullName, profilePicture, bio, notificationSettings.
    - Output: Updates User in database.
- **changePassword(newPassword)**: Boolean

    - Update password after verification.
    - Input: newPassword.
    - Output: True if updated, false if verification fails.
- **enrollInCourse(courseID)**: Boolean
    - Add course to student's enrolledCourses if eligible.
    - Input: courseID.
    - Output: True if enrolled, false if invalid.
- **getUserDetails()**: User
    - Retrieve all user attributes.
    - Input: None.
    - Output: User object.

3.5 Non-Functional Requirements

This section specifies the non-functional requirements for the Learning Management System (LMS), detailing system-wide quality attributes that guide its design, implementation, and testing. The requirements cover performance, reliability, availability, security, maintainability, and portability, ensuring the LMS meets stakeholder expectations without over-constraining the design.

3.5.1 Performance

3.5.1.1: **Page Load Time**

- **Description**: Pages load in <2s for 95% of requests.
- **Verification**: Test 1000 requests, 95% <2s.
- **Priority**: High.

3.5.1.2: **Concurrent User Support**

- **Description**: Support 500 concurrent users, response <2s for 95% of transactions.
- **Verification**: Simulate 500 users, 95% transactions <2s.
- **Priority**: High.

3.5.1.3: **File Upload Time**

- **Description**: Uploads ≤100 MB complete in <10s for 95% of cases.
- **Verification**: Test 100 uploads, 95% <10s.
- **Priority**: Medium.

3.5.2 Reliability

3.5.2.1: Mean Time Between Failures (MTBF)

- **Description**: MTBF ≥30 days for critical components.
- **Verification**: Monitor 6 months, failures <30 days apart.
- **Priority**: High.

3.5.2.2: Transaction Accuracy

- **Description**: 99.99% accuracy for transactions (e.g., enrollment, submission).
- **Verification**: Test 10,000 transactions, 99.99% error-free.
- **Priority**: High.

3.5.3 Availability

3.5.3.1: Scheduled Maintenance Window

- **Description**: Maintenance <2h/month, during 2 AM-4 AM.
- **Verification**: Review 6 months, maintenance <2h, in window.
- **Priority**: Medium.

3.5.3.2: System Uptime

- **Description**: 99.9% uptime, downtime <43 min/month (excluding maintenance).
- **Verification**: Monitor 3 months, downtime <43 min/month (e.g., Pingdom).
- **Priority**: High.

3.5.4 Security

3.5.4.1: Data Encryption

- **Description**: 100% sensitive data encrypted with AES-256 at rest.
- **Verification**: Audit confirms encryption compliance.
- **Priority**: High.

3.5.4.2: **Authentication Security**

- **Description**: 2FA for all logins, 99.9% verification <5s.
- **Verification**: Test 1000 logins, 99.9% <5s, no false positives.
- **Priority**: High.

3.5.4.3: **Regulatory Compliance**

- **Description**: 100% compliance with GDPR, FERPA for data handling.
- **Verification**: Audit verifies GDPR/FERPA compliance.
- **Priority**: High.

3.5.5 Maintainability

3.5.5.1: Code Modularity

- **Description**: 90% of components independently modifiable (low coupling, high cohesion).
- **Verification**: Analyze with SonarQube, 90% components modular.
- **Priority**: Medium.

3.5.5.2: Bug Fix Time

- **Description**: 95% of critical bugs fixed <24h.
- **Verification**: Track 100 bugs, 95% critical fixed <24h.
- **Priority**: High.

3.5.6 Portability

3.5.6.1: Cross-Platform Compatibility

- **Description**: Operates on 95% of devices (Chrome 100+, Firefox 95+, Safari 15+, iOS 15+, Android 10+).
- **Verification**: Test 20 device-browser combos, 95% error-free.
- **Priority**: High.

3.5.6.2: Cloud Provider Portability

- **Description**: 90% codebase reusable across AWS, Azure, Google Cloud.
- **Verification**: Deploy on AWS/Azure, 90% code unchanged.
- **Priority**: Medium.

3.6 Inverse Requirements

This section specifies the inverse requirements for the Learning Management System (LMS), identifying what the system explicitly should not do.

3.6.1: No Direct Video Conferencing

- **Description**: The LMS shall not provide built-in video conferencing capabilities for live classes or meetings, focusing instead on asynchronous course content delivery and management.
- **Rationale**: Including video conferencing would increase complexity and resource demands, diverting focus from core features like course enrollment and assignment submission. Users can use external tools (e.g., Zoom) for live interactions.
- **Verification**: Inspect the system codebase and interface to confirm no video conferencing modules or APIs are implemented.
- **Priority**: High

3.6.2: No Unauthenticated Access to Sensitive Data

- **Description**: The LMS shall not allow unauthenticated users to access sensitive data, such as course materials, grades, or user profiles, under any circumstances.
- **Rationale**: Ensuring data privacy and compliance with GDPR and FERPA requires strict access controls, preventing unauthorized access to protect user trust.
- **Verification**: Test 100 attempts to access sensitive data (e.g., grades, materials) without valid credentials, ensuring 100% are blocked with an error message.
- **Priority**: High

3.6.3: No Offline Mode

- **Description**: The LMS shall not support offline mode functionality, requiring an active internet connection for all user interactions, such as accessing course materials, submitting assignments, or viewing grades.
- **Rationale**: Offline mode would require complex data synchronization and storage mechanisms, increasing development effort and security risks, which is outside the project's scope focused on cloud-based, online access.
- **Verification**: Test 100 attempts to access LMS features (e.g., course materials, submissions) without an internet connection, ensuring 100% result in an error message (e.g., "Internet connection required").
- **Priority**: High

3.7 Design Constraints

This section specifies the design constraints imposed on the Learning Management System (LMS) software project.

3.7.1: GDPR and FERPA Compliance

- Description: Comply with GDPR/FERPA using AES-256 encryption and user consent.
- Rationale: Legally required, restricts data handling.
- Verification: Audit confirms 100% compliance.
- Priority: High.

3.7.2: Cloud-Based Architecture

- Description: Use cloud (e.g., AWS) for 500 users, no on-premises.
- Rationale: Ensures scalability within resource limits.
- Verification: Deploy on cloud, support 500 users (<2s response).
- Priority: High.

3.7.3: Six-Month Development Timeline

- Description: Complete core features by November 25, 2025.
- Rationale: Tight timeline limits scope.
- Verification: Deploy core features by deadline.
- Priority: High.

3.7.4: Open-Source or Low-Cost Tools

- Description: Use open source/low-cost tools (e.g., MySQL, React), no premium software.
- Rationale: Budget constraints require affordability.
- Verification: Confirm costs < $100/month.
- Priority: High.

3.8 Logical Database Requirements

This section specifies the logical database requirements for the Learning Management System (LMS), detailing the use of a database and the requirements for data formats, storage capabilities, data retention, data integrity, and related aspects. The requirements ensure the database supports the system's functionality while adhering to constraints like GDPR, FERPA, and cloud-based architecture.

Database usage

The LMS will use a relational database to store and manage all critical data, including user profiles, course details, assignments, grades, and system logs. The database will be hosted on a cloud-based platform (e.g., AWS RDS, Azure SQL).

3.8.1: Relational Database Model

- **Description**: The LMS shall use a relational database management system (e.g., MySQL, PostgreSQL) to organize data into tables with defined relationships (e.g., one-to-many

between Course and Student), supporting SQL queries for data retrieval and manipulation.

- **Rationale**: A relational model ensures structured storage and efficient querying for complex relationships, such as users enrolled in multiple courses.
- **Verification**: Validate the database schema includes tables (e.g., Users, Courses, Assignments) with correct relationships and supports 100 SQL queries with expected results.
- **Priority**: High

3.8.2: Data Formats

- **Description**: The database shall store data in standardized formats: Strings (UTF-8 for text, max 1000 characters for descriptions), Integers (e.g., enrollment limits, 0-1000), Dates (YYYY-MM-DD for start/end dates), and BLOBs (for file uploads like PDFs, ≤100 MB).
- **Rationale**: Standardized formats ensure compatibility, efficient storage, and support for internationalization.
- **Verification**: Test 100 data entries across all tables, ensuring 100% adherence to specified formats.
- **Priority**: High

3.8.3: Storage Capabilities

- **Description**: The database shall support storage of at least 10,000 user records, 1,000 courses, 100,000 assignments, and 1,000,000 system logs, with a minimum of 100 GB initial storage capacity, scalable to 1 TB.
- **Rationale**: These capacities accommodate a medium-sized institution with growth potential, aligning with cloud scalability requirements.
- **Verification**: Deploy the database with 100 GB storage and simulate storing maximum records (e.g., 10,000 users), ensuring no performance degradation; test scaling to 1 TB.
- **Priority**: High

3.8.4: Data Retention

- **Description**: The LMS shall retain user data (e.g., profiles, grades) for at least 5 years after account inactivity, course data (e.g., materials, assignments) for 3 years after course completion, and system logs for 1 year, unless users request deletion per GDPR.
- **Rationale**: Retention periods balance institutional needs with GDPR/FERPA compliance, allowing data deletion upon user request.
- **Verification**: Test retention by creating records and verifying they persist for specified periods (e.g., 5 years for user data); test user-initiated deletion completes within 24 hours.
- **Priority**: High

3.8.5: Data Encryption

- **Description**: All sensitive data (e.g., user passwords, grades, payment transaction IDs) stored in the database shall be encrypted using AES-256 at rest, with encryption keys managed securely via the cloud provider.
- **Rationale**: Encryption is required for GDPR/FERPA compliance and data security, limiting storage to secure configurations.
- **Verification**: Audit the database to confirm 100% of sensitive data is encrypted with AES-256; test key management ensures secure access.
- **Priority**: High

3.8.6: Data Integrity

- **Description**: The database shall maintain 99.99% data integrity for all transactions (e.g., enrollment, submission, grading), using constraints and ACID transactions to prevent data loss or corruption.
- **Rationale**: Integrity ensures reliable operations and trust in system data, critical for academic records.
- **Verification**: Execute 10,000 transactions (e.g., 5000 enrollments, 5000 submissions) and verify 99.99% complete without data loss or corruption via database audits.
- **Priority**: High

3.8.7: Backup and Recovery

- **Description**: The database shall support automated daily backups with a retention period of 30 days and allow full recovery within 1 hour of a failure, maintaining 99.9% data availability.
- **Rationale**: Backups ensure data recovery in case of failures, supporting system availability requirements.
- **Verification**: Test automated backups occur daily, retain for 30 days, and recover a 100 GB database in <1 hour with no data loss.
- **Priority**: High

3.9 Other Requirements

This section serves as a catchall for additional requirements for the Learning Management System (LMS) that do not fit into previous sections (e.g., external interfaces, functional, non-functional, database).

3.9.1: System Documentation

- **Description**: Include user/developer documentation in UI, updated per release, covering 100% core features.
- **Rationale**: Supports usability, maintainability.
- **Verification**: Audit confirms 100% feature coverage, UI accessibility.
- **Priority**: Medium.

3.9.2: Internationalization Support

- **Description**: UI localization for English/Spanish, 100% text translatable.
- **Rationale**: Broadens accessibility for international users.
- **Verification**: Test 100 UI elements for translatability.
- **Priority**: Medium.

4. Analysis Models

This section lists and describes the analysis models used to develop the requirements specified in the Software Requirements Specification (SRS) for the Learning Management System (LMS). The models—Sequence Diagrams, Data Flow Diagrams (DFD), and State-Transition Diagrams (STD)—provide structured representations of system behaviour, data flow, and state changes, ensuring clarity and traceability.

4.1 Sequence Diagrams

4.1.1: Sequence diagram: Enroll in a course

Introduction: This sequence diagram models the interaction between actors and system components during the course enrollment process, capturing the sequence of messages exchanged to achieve successful enrollment. It supports the analysis of functional requirements and use cases by illustrating the dynamic behavior of the system, ensuring all steps are accounted for, including error handling.

Narrative description: The diagram involves the student (actor), LMS Interface, Authentication Service, Course Object, Database, and Payment Gateway. The sequence begins with the student logging into the LMS Interface, which sends credentials to the Authentication Service for verification. Upon successful authentication, the student requests the course catalog, and the LMS Interface retrieves available courses from the Database via the Course Object. The student selects a course and initiates enrollment. For premium courses, the LMS Interface redirects to the Payment Gateway, which processes the payment and returns a success or failure response. If payment is successful or the course is free, the Course Object checks enrollment eligibility (e.g., course status, limit) and updates the Database with the Student's enrollment. The Database confirms the update, and the LMS Interface sends a confirmation notification to the Student. Error scenarios (e.g., invalid credentials, course full, payment failure) are modeled with alternative flows, triggering appropriate error messages. The diagram ensures all inputs, processing, outputs, and error handling from FR-1 and UC-1 are covered, with interactions aligned with the Course class functions (e.g., enrollStudent).

4.1.2: Sequence Diagram: Submit assignment

Introduction: This sequence diagram models the interaction between actors and system components during the assignment submission process, detailing the message flow to ensure a

student's submission is processed correctly. It supports the analysis of functional requirements and use cases by visualizing dynamic interactions, including validation and error handling.

Narrative description: The diagram involves the Student (actor), LMS Interface, Authentication Service, User Object, Assignment Object, Database, and Notification Service. The Student logs into the LMS Interface, which verifies credentials via the Authentication Service. Upon successful login, the Student navigates to the course's assignment section, and the LMS Interface retrieves open assignments from the Database via the Assignment Object. The Student selects an assignment, uploads a file (e.g., PDF, ≤100 MB), and optionally adds a comment. The LMS Interface validates the file (format, size, security) through the Assignment Object, which then stores the file and comment in the Database, updating the assignment status to "Submitted." The Database confirms the update, and the LMS Interface triggers the Notification Service to send a confirmation to the Student and an alert to the Instructor. Error scenarios (e.g., invalid file, assignment closed, duplicate submission) are modeled with alternative flows, displaying appropriate error messages. The diagram aligns with FR-2 and UC-2, incorporating User class functions (e.g., sendNotification) and ensuring file upload performance (NFR-3).

4.2 Data Flow Diagrams (DFD)

4.2.1: Data flow diagram: level 0 (context level)

Introduction: This Level 0 DFD (context diagram) provides a high-level view of the LMS, showing data flows between the system and external entities (actors and services). It supports the analysis of data interactions across all requirements, illustrating the system's boundaries and external interfaces.

Narrative description: The LMS is depicted as a single process interacting with external entities: Student, Instructor, Admin, Authentication Service, Payment Gateway, and Notification Service.

- Student: Sends credentials, course selections, assignment files, and support tickets to the LMS; receives course data, notifications, and support responses.
- Instructor: Sends course materials, grades, and support tickets; receives assignment submissions and notifications.
- Admin: Sends system configurations (e.g., course creation, user management); receives audit logs and system reports.
- Authentication Service: Receives credentials from the LMS; sends verification results.
- Payment Gateway: Receives payment details for courses; sends transaction confirmations.
- Notification Service: Receives notification requests; sends emails or in-app messages to users.
  Data flows include credentials, course data, files, payments, notifications, and logs,

aligning with FR-1 (enrollment), FR-2 (submission), and DC-6 (external integrations). The diagram ensures all major inputs/outputs from use cases (UC-1, UC-2) are represented.

4.2.2: Data flow diagram: level 1

Introduction: This Level 1 DFD decomposes the LMS into key processes, showing data flows between internal processes, data stores, and external entities. It supports detailed analysis of functional requirements and use cases by modeling how data is processed within the system.

Narrative description: The diagram includes processes: **Authenticate User**, **Manage Courses**, **Submit Assignment**, **Manage Notifications**, and **Handle Support Tickets**, interacting with data stores (Users, Courses, Assignments, Logs) and external entities (Student, Instructor, Admin, Authentication Service, Payment Gateway, Notification Service).

- **Authenticate User**: Receives credentials from Student/Instructor, validates via Authentication Service, and stores login logs in Logs.
- **Manage Courses**: Receives course selections or creation requests, updates Courses store (e.g., enrollment via Course.enrollStudent), and processes payments via Payment Gateway.
- **Submit Assignment**: Receives files/comments from Student, validates and stores in Assignments, updates Users (e.g., enrolledCourses), and triggers notifications.
- **Manage Notifications**: Sends confirmation/alerts via Notification Service, storing notification logs in Logs.
- **Handle Support Tickets**: Receives tickets, stores in Logs, and sends responses to users. Data stores (Users, Courses, Assignments) align with DBR-1 (relational model) and class attributes (Course, User). Flows cover all inputs/outputs from FR-1 and FR-2, supporting UC-1 and UC-2 steps and ensuring database integrity (DBR-5).

4.3 State-Transition Diagrams (STD)

4.3.1: State-Transition diagram: Assignment lifecycle

Introduction: This STD models the lifecycle of an Assignment object, illustrating its states and transitions from creation to grading and archiving. It supports the analysis of assignment-related requirements by defining how an assignment progresses through various states, ensuring alignment with submission, grading, and data retention processes. The model clarifies the dynamic behavior of assignments, supporting functional requirements and use cases related to assignment submission.

Narrative description: The Assignment object progresses through the following states: **Draft**, **Open**, **Closed**, **Graded**, and **Archived**.

- **Draft**: The assignment is created by the instructor (e.g., via a createAssignment function, specifying title, due date, and requirements) but is not yet published for student access. The instructor can modify details (e.g., description, due date) in this state.
- **Open**: The assignment is published, allowing enrolled students to submit files and comments (via submitAssignment). Submissions are accepted until the due date or manual closure by the instructor. The system validates submissions (e.g., file size ≤100 MB, per UC-2).
- **Closed**: The assignment reaches its due date or is manually closed by the instructor, prohibiting further submissions. Instructors can begin grading submissions in this state.
- **Graded**: All submissions are graded, and grades are finalized and recorded in the database. Students can view their grades, and no further changes to submissions or grades are allowed.
- **Archived**: The assignment data (e.g., submissions, grades) is moved to long-term storage after 3 years from closure (per DBR-4), retaining data for compliance but restricting access to read-only for authorized users (e.g., admins).
  Transitions are triggered by events or actions:
- **Draft to Open**: Instructor publishes the assignment.
- **Open to Closed**: Due date passes or instructor closes manually.
- **Closed to Graded**: Instructor completes and finalizes grading.
- **Graded to Archived**: 3 years elapse since closure.
- **Draft to Archived**: Instructor deletes unpublished assignment (rare case).
  The diagram ensures the Assignment object supports FR-2 and UC-2 by managing submission eligibility (only in “Open” state), notification triggers (e.g., submission confirmation), and data retention (DBR-4). It also aligns with User class functions (e.g., sendNotification for submission and grading alerts).

5. Change Management Process

This section outlines the process for updating the Software Requirements Specification (SRS) for the Learning Management System (LMS) when project scope or requirements change.

5.1 Change Management Process Description

The change management process for the LMS SRS follows a structured approach to handle updates due to scope or requirement changes, ensuring stakeholder consensus and traceability. The process includes submission, evaluation, approval, and implementation phases, with clear roles, tools, and timelines to maintain project control within the 6-month development timeline (SRS Section 3.7, DC-5).

5.1.1 Roles and Responsibilities

- **Stakeholders**: Sponsor, end-users (students, instructors, admins), developers; submit changes.
- **Change Requestor**: Any stakeholder identifying a change need.
- **Change Control Board (CCB)**: Project manager, lead developer, stakeholder rep, QA lead; evaluates/approves changes.
- **Project Manager**: Facilitates process, communicates decisions.
- **SRS Document Owner**: Updates SRS maintains version control.

5.1.2 Change Submission Process

- **Who**: Any stakeholder (e.g., student, developer).
- **Means**: Submit via Change Request Form (Jira/GitHub Issues) with:
  - Requestor details (name, role, contact).
  - Change description (e.g., new feature).
  - Rationale (e.g., user feedback).
  - Impact (scope, timeline, budget).
  - Priority (High/Medium/Low).
- **Process**: Logged with unique ID. Project manager reviews for completeness <24h, rejects incomplete forms.
- **Verification**: Confirm 100% submissions logged with ID.

5.1.3 Change Evaluation Process

- **Initial Review**: Project manager assigns to CCB <48h; CCB assesses feasibility (timeline, budget).
- **Impact Analysis**: Development team evaluates:
  - SRS sections affected (e.g., FR-2).
  - Schedule (e.g., 2 weeks added).
  - Resources (e.g., AWS costs).
  - Risks (e.g., core feature delay).
  - QA assesses testing needs. Completed <5 days.
- **Feedback**: CCB collects stakeholder input <2 days.
- **Verification**: Confirm analysis documented in form.

5.1.4 Change Approval Process

- **CCB Decision**: Meets <7 days, reviews request/analysis. Criteria:
  - Aligns with goals (e.g., no scope creep, IR-1).
  - Feasible within timeline/budget (DC-4, DC-5).
  - Minimal disruption to critical requirements.
- **Outcomes**: Approved, Rejected, Deferred (majority vote).
- **Documentation**: Decision, rationale, votes logged in Jira/GitHub.
- **Notification**: Requestor/stakeholders informed via email/platform <24h.
- **Verification**: Confirm 100% decisions documented, communicated.

5.1.5 Change Implementation Process

- **SRS Update**: Document Owner updates SRS (e.g., new OR-6), increments version (e.g., v1.1), links to CR-ID, stores in GitHub.
- **Review**: CCB verifies updates <3 days; QA checks traceability.
- **Communication**: Updated SRS distributed via email/platform.
- **Timeline**: Process completes <15 days for 95% requests.
- **Verification**: Confirm updates reflect changes, versioned correctly.

5.1.6 Tools and Tracking

- **Platform**: Jira/GitHub Issues for tracking.
- **Version Control**: GitHub for SRS updates.
- **Communication**: Email/Slack for notifications.
- **Audit Trail**: 100% requests logged (OR-4).
- **Verification**: Audit confirms complete trail.

Verification

- Simulate 10 requests, ensure:
  - 100% logged with ID.
  - 95% complete <15 days.
  - Updates versioned, communicated.
  - 100% audit trail in Jira/GitHub.

A. Appendices

This section provides additional information to support the Software Requirements Specification (SRS) for the Learning Management System (LMS). The appendices include supplementary materials that enhance understanding of the project context, requirements, and development process but are not considered part of the SRS's formal set of requirements. They serve as reference points for stakeholders and developers, offering insights into the project's conceptual foundation and stakeholder interactions.

Statement on Appendices

The information contained within the appendices (A.1 and A.2) is not part of the SRS's formal requirements. These appendices are provided for informational purposes only, to offer context, background, and supporting documentation. They do not impose additional obligations or constraints on the system's design or implementation beyond the requirements specified in Sections 3.2 through 3.9 and the analysis models in Section 4.

A.1 Appendix 1: Initial Conceptual Document for the LMS Project

Introduction: This appendix contains the initial conceptual document drafted at the start of the LMS project, outlining the vision, objectives, and high-level scope. It provides context for the requirements specified in the SRS and helps stakeholders understand the project's intent.

Content Description:

The conceptual document includes:

- **Project Vision:** To develop a cloud-based LMS that supports asynchronous course delivery, assignment submission, and user management for a medium-sized academic institution, targeting 10,000 students and 1,000 courses.
- **Objectives:**
  - Enhance student and instructor experience with a user-friendly interface.
  - Ensure scalability for 500 concurrent users (SRS Section 3.5.1, NFR-2).
  - Comply with GDPR and FERPA for data privacy (SRS Section 3.7, DC-1).
  - Complete development within 6 months using open-source tools (SRS Section 3.7, DC-4, DC-5).
- **High-Level Scope**: Core features include course enrollment, assignment submission, and grade management, excluding real-time video conferencing or offline mode (SRS Section 3.6, IR-1, IR-5).
- **Stakeholders**: Academic institution (sponsor), students, instructors, admins, and student development team.

- **Assumptions**: Stable cloud infrastructure (e.g., AWS), access to free/low-cost tools, and stakeholder availability for feedback.
  This document was used to align initial stakeholder expectations and informed the development of functional requirements (e.g., FR-1, FR-2 in SRS Section 3.2) and constraints (e.g., DC-2, DC-4 in SRS Section 3.7). It is stored in the project's documentation repository (e.g., GitHub) for reference.

Purpose:

Provides historical context for the project's goals and scope, aiding stakeholders in understanding the rationale behind SRS requirements. It is not binding but supports traceability to the project's inception.

A.2 Appendix 2

Introduction: This appendix is with a comprehensive stakeholder feedback analysis report, compiled after multiple feedback sessions with the academic institution, students, and instructors. The report summarizes user input on the LMS's proposed features and prototype, providing valuable insights that informed the SRS requirements.

Content Description:

The report, finalized on May 10, 2025, aggregates feedback from three feedback sessions (April 20, April 27, and May 5, 2025) with stakeholders, including 5 student representatives, 3 instructors, and 2 institution representatives. Key points include:

- **Feedback on Features**:
  - **Students**: Requested intuitive navigation for enrollment and assignment submission, with confirmation notifications (SRS Section 3.3.1, UC-1; Section 3.3.2, UC-2). Emphasized mobile compatibility (SRS Section 3.7, DC-3) and fast page loads (<2 seconds, SRS Section 3.5.1, NFR-1).
  - **Instructors**: Prioritized clear assignment grading workflows and automated submission alerts (SRS Section 3.2.2, FR-2; Section 3.9, OR-4). Suggested detailed user manuals (SRS Section 3.9, OR-2).
  - **Institution**: Stressed GDPR/FERPA compliance (SRS Section 3.7, DC-1), audit logging for accountability (SRS Section 3.9, OR-4), and no offline mode to simplify development (SRS Section 3.6, IR-1).
- **Feedback on Prototype**:
  - **Positive**: Clean UI, easy course browsing.
  - **Concerns**: Initial login delays (addressed in SRS Section 3.5.1, NFR-2), need for Spanish language option (SRS Section 3.9, OR-3).
  - **Suggestions**: Add helpdesk access in UI (SRS Section 3.9, OR-1), ensure WCAG 2.1 accessibility (SRS Section 3.9, OR-5).

- **Prioritization**:
  - **High**: Core features (enrollment, submission), compliance, accessibility.
  - **Medium**: Internationalization, documentation, helpdesk.
- **Action Taken**: Feedback shaped use cases (UC-1, UC-2), non-functional requirements (NFR-1, NFR-2), and other requirements (OR-1 to OR-5). Prototype login delays were mitigated by optimizing authentication (SRS Section 3.3.3, UC-3).
  The report is stored in the project's GitHub repository and includes table mapping feedback to SRS sections for traceability.

Purpose:
Provides a consolidated view of stakeholder feedback, clarifying the origins of SRS requirements and supporting future refinements. It is non-binding but enhances transparency.

- **Verification**: Confirm report exists in repository, accurately summarizes feedback from sessions, and aligns with SRS requirements without introducing conflicts.